Plan: Entropy-Heuristic Top-K Cycle Selection for Vertex-Uniform M_e

2026-05-10

Motivation

The smoke test from the prior task (`reports/2026-05-10-abb-global-topk.md`) found that *global* top-K ABB delivers a 25× wall-time reduction on Epinions but *does not* help HSiKAN training: at K=10 000 all retained cycles are score-1.0 (all-negative balanced) the M_e matrix has no score variance and no per-vertex coverage in the long tail. Single-seed AUC dropped from 0.642 (per-vertex m=128 baseline) to 0.575 (global ABB at the matched abbreviated config).

The per-vertex top-m enumerator works because it gives *every* vertex its top-m cycles, producing a vertex-uniform M_e. The cost is the $O(n_{\text{vertices}} \cdot m)$ per-vertex heap traffic and the BFS pre-pass that dominates 66% of cycles in the post-CSR profile.

This plan proposes a third path: select cycles to *maximise the entropy of the per-vertex incidence distribution*. Vertices that are already well-covered contribute less to the next cycle's score; rare vertices contribute more. The result is vertex-uniform M_e *by construction*, paired with branch-and-bound pruning to keep wall time low.

Definitions

Let $C \subseteq \mathcal{C}_k$ be the current set of accepted k-cycles. Define the per-vertex incidence count $c_v = |\{c \in C : v \in c\}|$ and the empirical incidence distribution

$$p_v = \frac{c_v}{\sum_w c_w}.$$

The Shannon entropy of the incidence distribution is

$$H(C) = -\sum_v p_v \log p_v,$$

maximised at $p_v = 1/|\{v : c_v > 0\}|$ (uniform over covered vertices).

Goal: given a budget K, select $C^* \subseteq \mathcal{C}_k$ with $|C^*| = K$ that approximately maximises $H(C^*)$, possibly subject to a pruner $P : \mathcal{C}_k \to \{\text{accept, reject}\}$.

Approach

Greedy entropy-gain scoring

For a candidate cycle $c \in \mathcal{C}_k$ relative to the current C, the marginal entropy gain is

$$\Delta H(c \mid C) = H(C \cup \{c\}) - H(C).$$

A standard greedy max-coverage argument gives a $(1 - 1/e)$ approximation to the optimum if H is monotone submodular (approximately true for vertex-coverage entropy until saturation).

State-dependent scorer

Add a new trait `StatefulScorer` that takes the running per-vertex counts c and the candidate cycle:

```
pub trait StatefulScorer: Sync + Send {
    /// Score this cycle given the current per-vertex incidence
    /// counts.  For entropy-heuristic: returns $\Delta$H(c | C).
    fn score(&self, vs: &[u32], signs: &[i8], counts: &[u32],
             total: u64) -> f64;
    /// Update the running counts to include this cycle.
    fn update(&self, vs: &[u32], counts: &mut [u32], total: &mut u64);
    /// Upper bound on score given partial state.  For entropy:
    /// use rare-vertex bound  score cannot exceed the entropy
    /// contribution of the LEAST-covered vertex among the cycle's
    /// committed prefix.
    fn upper_bound(&self, prefix_vs: &[u32], k_remaining: usize,
                   counts: &[u32], total: u64) -> f64;
}
```

Two concrete scorers

1. `EntropyGainScorer`: $\Delta H(c \mid C)$ as above. State-dependent, monotonically decreasing as C grows.

2. `InverseDegreeScorer` (state-independent baseline):

$$\text{score}(c) = \sum_{v \in c} \frac{1}{\sqrt{c_v + 1}},$$

   where the $\sqrt{}$ smooths the contribution. No state update needed across DFS recursion (just read c); cheaper to bound.

Branch-and-bound on entropy gain

For `EntropyGainScorer`, the partial-path UB is:

$$\text{UB}(\text{prefix}, k_{\text{rem}}, c, \text{total}) = \Delta H_{\text{prefix}} + k_{\text{rem}} \cdot \max_v [\Delta H_v]$$

where ΔH_v is the entropy contribution of adding vertex v to the running cycle, taking the maximum over all reachable future vertices (loose; tighter via BFS-distance-restricted reachable set).

If `UB` $\leq$ heap min, prune the subtree. Threshold rises as the heap fills with high-ΔH cycles ABB fires progressively.

Per-thread state for rayon

A subtlety: `StatefulScorer` maintains running counts. In the rayon-parallel variant, each fold task gets its own count vector; entropy is computed locally; reduce merges by re-scoring. This trades exactness (a cycle that looks high-ΔH on thread 1 might be redundant after thread 2's set is merged in) for parallelism. Acceptable for a heuristic: the global greedy bound is already approximate.

Scope and goal

Add an entropy-heuristic top-K enumerator alongside the existing ABB / per-vertex paths. Specifically:

- New trait `StatefulScorer` in `hymeko_graph::topk_cycles`.
- Concrete impls: `EntropyGainScorer`, `InverseDegreeScorer`.
- New entry points: `enumerate_top_k_cycles_entropy` (sequential) and `enumerate_top_k_cycles_par_entr` (rayon).
- Builder method: `TopKBuilder::entropy(k_keep, scorer)`.
- PyO3 binding: `enumerate_top_k_cycles_signed_entropy_rs`.
- Python route: `HSIKAN_TOPK_MODE=entropy`.

Goal: on Epinions k=4, abbreviated training config (c3+c4, h=4, 20 epochs):

- AUC within ±0.02 of per-vertex m=128 baseline (0.642 on this seed). This is the *information sufficiency* test.
- Wall time ≤ 30 s (vs 148.6 s for per-vertex baseline). This is the *speedup* test.

The 5-seed paired AUC validation per the "`n-seed before paper-headline`" memory rule is gated on the smoke-test passing first.

Affected files

- `hymeko_graph/src/topk_cycles.rs` — additive: `StatefulScorer` trait + 2 impls + 2 enumerate-entry-points + private `dfs_entropy` helper, ∼ 400 LOC.
- `hymeko_graph/src/lib.rs` — re-export new symbols.
- `hymeko_graph/tests/entropy_topk.rs` — new integration: per-vertex coverage uniformity vs ABB; AUC-style diversity proxy on a synthetic graph; admissibility of the entropy UB.
- `hymeko_py/src/cycles.rs`, `hymeko_py/src/lib.rs` — new PyO3 binding.
- `signedkan_wip/src/n_tuples.py` — route `HSIKAN_TOPK_MODE=entropy` through the new binding.

CORE.YAML items touched

None. `hymeko_graph` and `hymeko_py` are not in the lockdown list; `tests/**` is in `allowlist`. No new dependencies (entropy computed via `f64::ln` on local count buffers).

Test strategy

Unit (lib)

- `StatefulScorer::EntropyGainScorer` returns 0 on a fresh empty C for cycles already seen, > 0 for cycles that add new vertices.
- `InverseDegreeScorer` is monotonic in counts.
- Entropy UB is admissible: for 1 000 random partial paths, every reachable completion's actual ΔH is ≤ UB.

Integration

- On a 200-vertex Erdős–Rényi fixture, the entropy- heuristic output's per-vertex coverage variance is at most 1.5× the per-vertex top-m baseline's coverage variance (i.e. the entropy path achieves comparable uniformity).
- The output cardinality matches K (or all reachable cycles, if fewer).
- Comparison with global ABB: entropy-heuristic produces a lower-mean-score, higher-coverage cycle set.

Performance

- Criterion bench on 5 000-vertex synthetic graph, k=4, K=10 000, balance pruner: *informational* only no asserted budget, since the goal is information quality, not raw speed.
- Real-data smoke: 1-seed Epinions training as in the prior task's smoke test, with `HSIKAN_TOPK_MODE=entro` `HSIKAN_TOPK_K=10000`. Report wall time + AUC vs the two baselines (per-vertex, global ABB).

HSiKAN promotion gate

If the smoke test passes (AUC within ±0.02 of per-vertex baseline AND wall ≤ 30 s), trigger 5-seed paired validation:

- 5 seeds » {per-vertex m=128, entropy K=10 000, entropy K=100 000} on Epinions (the production-relevant dataset). Production config: c2,c3,c4,c5,w2,w3, h=4, 80 epochs, balance pruner.
- Acceptance: paired-mean AUC within ±0.005 of per-vertex baseline, paired-σ within ±1.0σ. Below threshold ⇒ entropy-heuristic ships as a research tool, not production.

Performance budget

Metric	Per-vertex baseline	Entropy budget	Status
1-seed Epinions wall (full pipeline)	148.6 s	≤ 30 s	smoke gate
1-seed Epinions AUC (c3+c4, h=4, 20 ep)	0.642	≥ 0.622	smoke gate
5-seed Epinions paired AUC (production config)	TBD	within ±0.005	promotion gate
Output cardinality	$= K$	$= K$	exact
Per-vertex coverage variance vs baseline	1.0	≤ 1.5×	integration test
16 GB ğ4 cap	under	under	—

Rollback path

Strictly additive `TopKBuilder::entropy` is a new method, `StatefulScorer` is a new trait, the entropy-aware enumerators are new entry points. No existing behavior changes. Single revert restores prior state; the per-vertex baseline and global ABB tools remain.

Halt condition: if the smoke-test AUC drops below the ±0.02 band, the entropy-heuristic ships as a research artefact only and is not promoted to HSiKAN production; the `plan B` fallback (degree-adaptive m_v in the existing per-vertex path) is re-opened as a follow-up plan.

Architecture diagram

See `plan.tikz` for the entropy-driven DFS state machine. See `plan.mmd` for the cycle accept/reject sequence and running-count update.